Order on the Go: Your On-Demand Food Ordering Solutions

Project Overview & Team Details

Project Title: Order on the Go: Your On-Demand Food Ordering Solutions

Team Members And Their Roles:

Member-1:- K S V Satya Sai - [Backend Development & Project Implementation & Execution]

Member-2:- G S Pradeepthi – [Frontend Development & Database Development]

Member-3:- J Yasasvee – [Project Setup And Configuration & Frontend Development]

Member-4:- K Siddarda – [Backend Development & Project Setup And Configuration]

Introduction

Introducing SB Foods, the cutting-edge digital platform poised to revolutionize the way you order food online. With SB Foods, your food ordering experience will reach unparalleled levels of convenience and efficiency.

Our user-friendly web app empowers foodies to effortlessly explore, discover, and order dishes tailored to their unique tastes. Whether you're a seasoned food enthusiast or an occasional diner, finding the perfect meals has never been more straightforward.

Imagine having comprehensive details about each dish at your fingertips. From dish descriptions and customer reviews to pricing and available promotions, you'll have all the information you need to make well-informed choices. No more second-guessing or uncertainty – SB Foods ensures that every aspect of your online food ordering journey is crystal clear.

The ordering process is a breeze. Just provide your name, delivery address, and preferred payment method, along with your desired dishes. Once you place your order, you'll receive an instant confirmation. No more waiting in long queues or dealing with complicated ordering processes – SB Foods streamlines it, making it quick and hassle-free.

SCENARIO: -

Late-Night Craving Resolution

Meet Lisa, a college student burning the midnight oil to finish her assignment. As the clock strikes midnight, her stomach grumbles, reminding her that she skipped dinner. Lisa doesn't want to interrupt her workflow by cooking, nor does she have the energy to venture outside in search of food.

Solution with Food Ordering App:

1. Lisa opens the Food Ordering App on her smartphone and navigates to the late-night delivery section, where she finds a variety of eateries still open for orders.

2. She scrolls through the options, browsing menus and checking reviews until she spots her favorite local diner offering comfort food classics.

3. Lisa selects a hearty bowl of chicken noodle soup and a side of garlic bread, craving warmth and satisfaction in each bite.

4. With a few taps, she adds the items to her cart, specifies her delivery address, and chooses her preferred payment method.

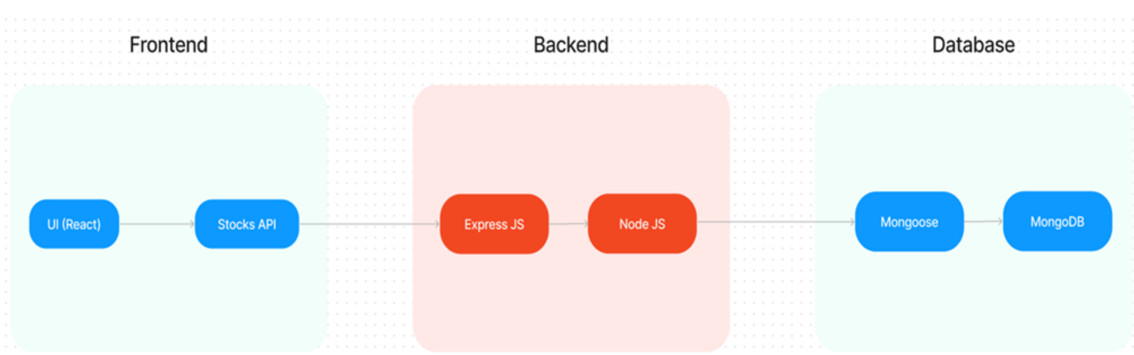
5. Lisa double-checks her order details on the confirmation page, ensuring everything looks correct, before tapping the "Place Order" button.

6. Within minutes, she receives a notification confirming her order and estimated delivery time, allowing her to continue working with peace of mind.

7. As promised, the delivery arrives promptly at her doorstep, and Lisa eagerly digs into her piping hot meal, grateful for the convenience and comfort provided by the Food Ordering App during her late-night study session.

This scenario illustrates how a Food Ordering App caters to users' needs, even during unconventional hours, by offering a seamless and convenient solution for satisfying late-night cravings without compromising on quality or convenience.

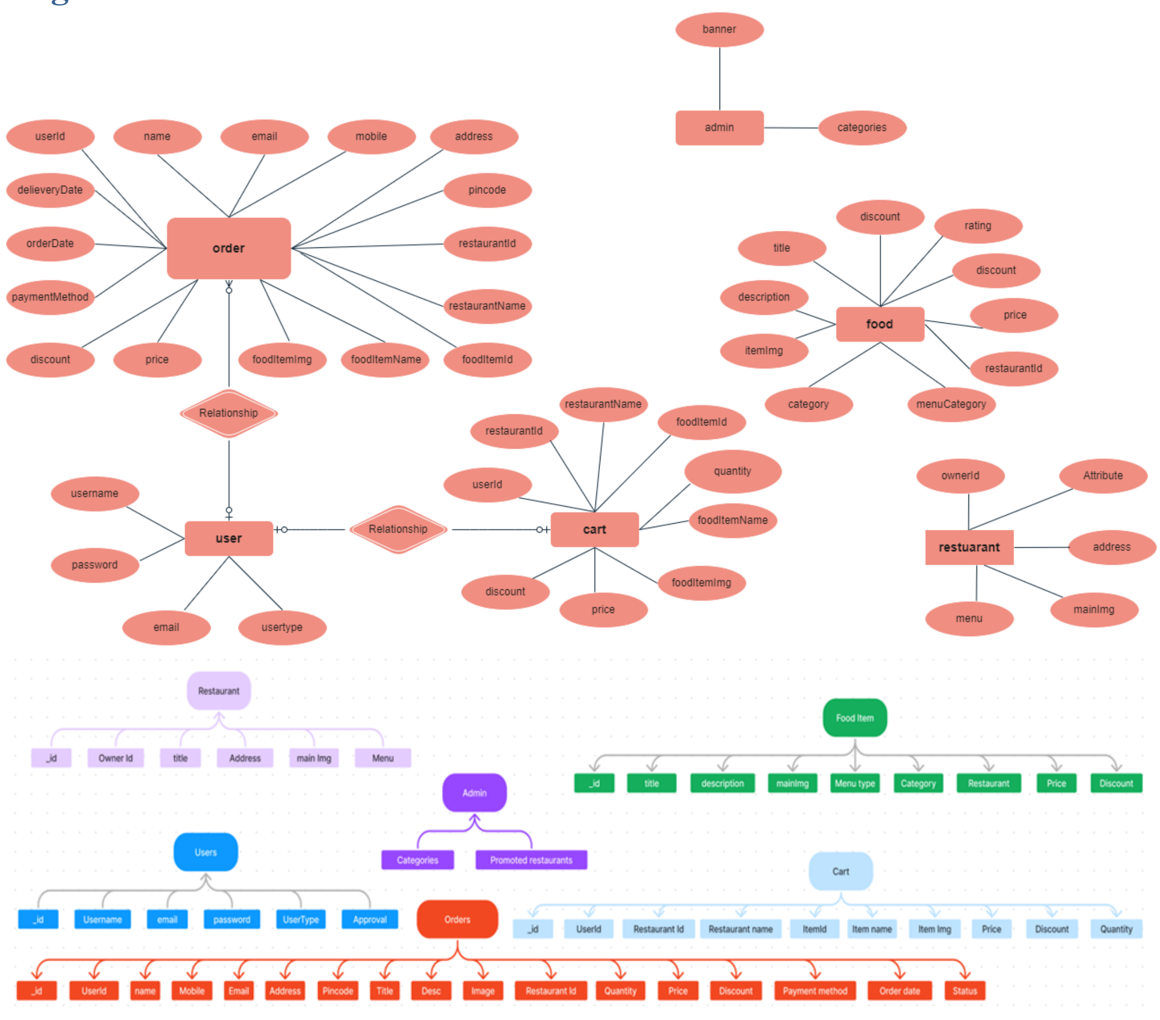
TECHNICAL ARCHITECTURE:



In this architecture diagram:

- The frontend is represented by the "Frontend" section, including user interface components such as User Authentication, Cart, Products, Profile, Admin dashboard, etc.,
- The backend is represented by the "Backend" section, consisting of API endpoints for Users, Orders, Products, etc., It also includes Admin Authentication and an Admin Dashboard.
- The Database section represents the database that stores collections for Users, Admin, Cart, Orders, and products.

ER-Diagram



The SB Foods ER-diagram represents the entities and relationships involved in an food ordering e-commerce system. It illustrates how users, restaurants, products, carts, and orders are interconnected. Here is a breakdown of the entities and their relationships:

User: Represents the individuals or entities who are registered in the platform.

Restaurant: This represents the collection of details of each restaurant in the platform. **Admin:** Represents a collection with important details such as promoted restaurants and Categories.

Products: Represents a collection of all the food items available in the platform.

Cart: This collection stores all the products that are added to the cart by users. Here, the elements in the cart are differentiated by the user Id.

Orders: This collection stores all the orders that are made by the users in the platform.

FEATURES:

1. **Comprehensive Product Catalog:** SB Foods boasts an extensive catalog of food items from various restaurants, offering a diverse range of items and options for shoppers. You can effortlessly explore and discover various products, complete with detailed descriptions, customer reviews, pricing, and available discounts, to find the perfect food for your hunger.

2. **Order Details Page**: Upon clicking the "Shop Now" button, you will be directed to an order details page. Here, you can provide relevant information such as your shipping address, preferred payment method, and any specific product requirements.

3. **Secure and Efficient Checkout Process:** SB Foods guarantees a secure and efficient checkout process. Your personal information will be handled with the utmost security, and we strive to make the purchasing process as swift and trouble-free as possible.

4. **Order Confirmation and Details:** After successfully placing an order, you will receive a confirmation notification. Subsequently, you will be directed to an order details page, where you can review all pertinent information about your order, including shipping details, payment method, and any specific product requests you specified.

In addition to these user-centric features, SB Foods provides a robust restaurant dashboard, offering restaurants an array of functionalities to efficiently manage their products and sales. With the restaurant dashboard, restaurants can add and oversee multiple product listings, view order history, monitor customer activity, and access order details for all purchases.

SB Foods is designed to elevate your online food ordering experience by providing a seamless and user-friendly way to discover your desired foods. With our efficient checkout process, comprehensive product catalog, and robust restaurant dashboard, we ensure a convenient and enjoyable online shopping experience for both shoppers and restaurants alike.

Application Flow

NODE.JS AND NPM:

- Node.js is a JavaScript runtime that allows you to run JavaScript code on the server-side. It provides a scalable platform for network applications.
- npm (Node Package Manager) is required to install libraries and manage dependencies.
- Download Node.js: Node.js Download
- Installation instructions: Installation Guide
- Run npm init to set up the project and create a package.json file.

EXPRESS.JS:

- Express.js is a web application framework for Node.js that helps you build APIs and web applications with features like routing and middleware.
- Install Express.js to manage backend routing and API endpoints.
- Install Express:
- Run npm install express

HTML, CSS, and JavaScript:

Basic knowledge of HTML for creating the structure of your app, CSS for styling,and JavaScript for client-side interactivity is essential.

Database Connectivity:

Use a MongoDB driver or an Object-Document Mapping (ODM) library like Mongoose to connect your Node.js server with the MongoDB database and perform CRUD (Create, Read, Update, Delete) operations.

Front-end Framework:

Utilize Angular to build the user-facing part of the application, including product listings, booking forms, and user interfaces for the admin dashboard.

Version Control:

Use Git for version control, enabling collaboration and tracking changes throughout the development process. Platforms like GitHub or Bitbucket can host your repository.

MONGODB:

- MongoDB is a NoSQL database that stores data in a JSON-like format, making it suitable for storing data like user profiles, doctor details, and appointments.
- Set up a MongoDB database for your application to store data.
- Download MongoDB: MongoDB Download
- Installation instructions: MongoDB Installation Guide

REACT.JS:

- React.js is a popular JavaScript library for building interactive and reusable user interfaces. It enables the development of dynamic web applications.
- Install React.js to build the frontend for your application.
- React.js Documentation: Create a New React App

Application Flow

User Flow

- Register for an account and log in.

- Browse products and add items to the cart.

- Enter delivery and payment details to complete the order.

- Track order history through the profile section.

Restaurant Flow

- Authenticate and request admin approval.

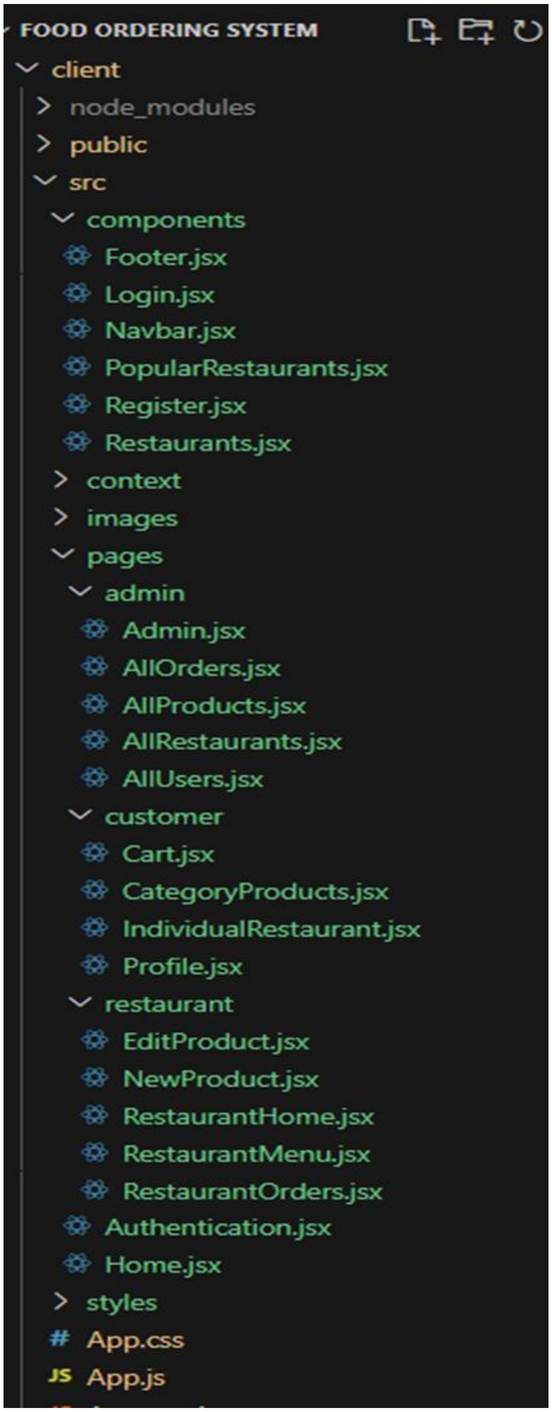
- List, update, and manage menu items.

Admin Flow

- Log in to the admin dashboard.

- Manage users, restaurants, products, and orders.

Project Structure



This structure assumes a React app and follows a modular approach. Here's a brief explanation of the main directories and files:

• src/components: Contains components related to the application such as, register, login, home, etc.,

• src/pages has the files for all the pages in the application.

Architecture

Frontend

1. Setup React Application:

- • Create a React app in the client folder.
- Install required libraries
- Create required pages and components and add routes.

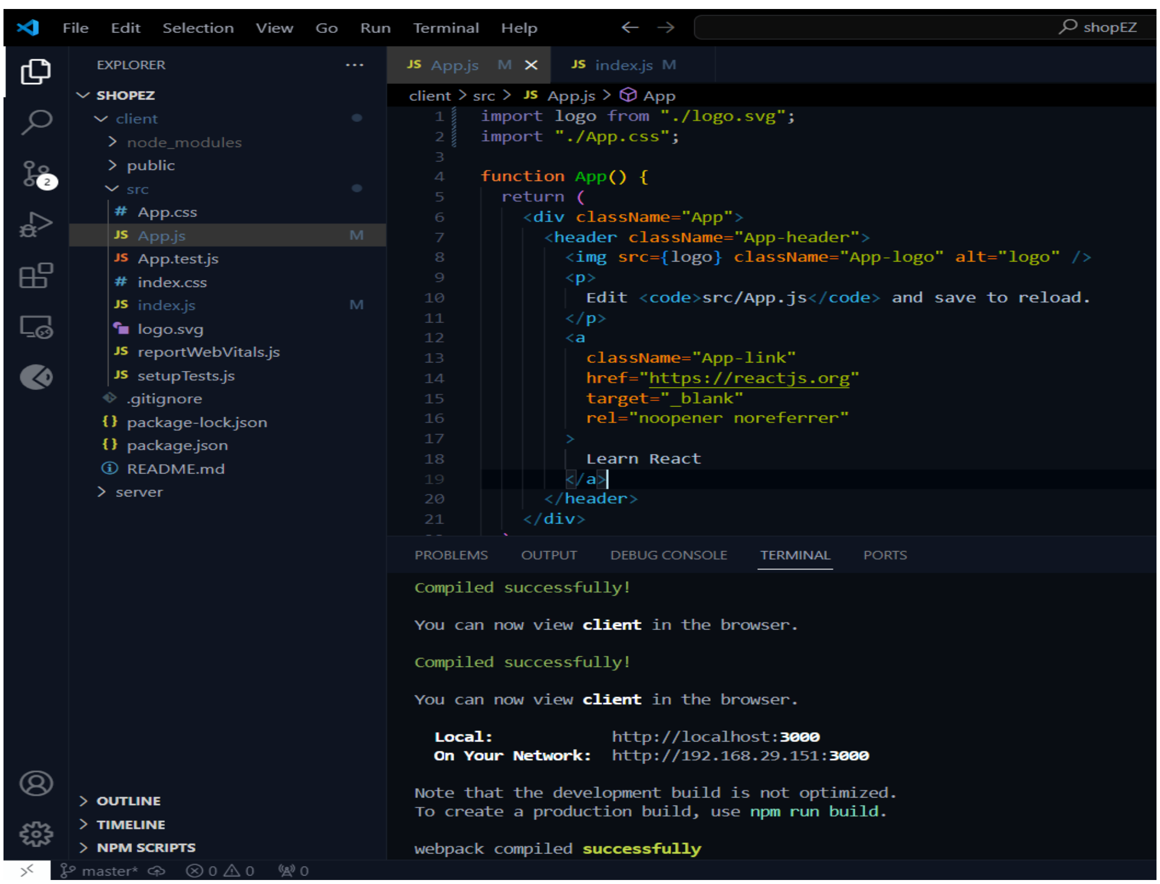
2.Design UI components:

- Create Components.
- Implement layout and styling.
- Add navigation.

3.Implement frontend logic:

- Integration with API endpoints.
- Implement data binding.

Reference Image:



Code Explanation:

Server setup:

Let us import all the required tools/libraries and connect the database.

```
import express from 'express'
import bodyParser from 'body-parser';
import mongoose from 'mongoose';
import cors from 'cors';
import bcrypt from 'bcrypt';
import {Admin, Cart, FoodItem, Orders, Restaurant, User } from './Schema.js'


const app = express();

app.use(express.json());
app.use(bodyParser.json({limit: "30mb", extended: true}))
app.use(bodyParser.urlencoded({limit: "30mb", extended: true}));
app.use(cors());

const PORT = 6001;

mongoose.connect('mongodb://localhost:27017/foodDelivery',{
    useNewUrlParser: true,
    useUnifiedTopology: true
}).then(()=>{
```

User Authentication:

- **Backend**

Now, here we define the functions to handle http requests from the client for authentication.

```
    app.post('/login', async (req, res) => {
        const { email, password } = req.body;
        try {
            const user = await User.findOne({ email });

            if (!user) {
                return res.status(401).json({ message: 'Invalid email or password' });
            }
            const isMatch = await bcrypt.compare(password, user.password);
            if (!isMatch) {
                return res.status(401).json({ message: 'Invalid email or password' });
            } else{
                return res.json(user);
            }
        } catch (error) {
          console.log(error);
          return res.status(500).json({ message: 'Server Error' });
        }
    });
```

```
index.js ×
server > index.js > then() callback > app.post('/login') callback
    app.post('/register', async (req, res) => {
        const { username, email, usertype, password , restaurantAddress, restaurantImage} = req.body;
        try {
            const existingUser = await User.findOne({ email });
            if (existingUser) {
                return res.status(400).json({ message: 'User already exists' });
            }
            const hashedPassword = await bcrypt.hash(password, 10);
            if(usertype === 'restaurant'){
                const newUser = new User({
                    username, email, usertype, password: hashedPassword, approval: 'pending'
                });
                const user =  await newUser.save();
                console.log(user._id);
                const restaurant = new Restaurant({ownerId: user._id ,title: username,
                            address: restaurantAddress, mainImg: restaurantImage, menu: []});
                await restaurant.save();
                return res.status(201).json(user);
            } else{
                const newUser = new User({
                    username, email, usertype, password: hashedPassword, approval: 'approved'
                });
                const userCreated = await newUser.save();
                return res.status(201).json(userCreated);
            }
        } catch (error) {
          console.log(error);
          return res.status(500).json({ message: 'Server Error' });
        }
    });
```

· **Frontend**

Login:

```
GeneralContext.js U ×
client > src > context > GeneralContext.js > GeneralContextProvider > register > then() callback
  const login = async () =>{
    try{
      const loginInputs = {email, password}
        await axios.post('http://localhost:6001/login', loginInputs)
        .then( async (res)=>{

          localStorage.setItem('userId', res.data._id);
            localStorage.setItem('userType', res.data.usertype);
            localStorage.setItem('username', res.data.username);
            localStorage.setItem('email', res.data.email);
            if(res.data.usertype === 'customer'){
                navigate('/');
            } else if(res.data.usertype === 'admin'){
                navigate('/admin');
            }
          }).catch((err) =>{
            alert("login failed!!");
            console.log(err);
          });
        }catch(err){
          console.log(err);
        }
      }
```

Logout:

```
GeneralContext.jsx U ×
client > src > context > GeneralContext.jsx > GeneralContextProvider > login >

  const logout = async () =>{

    localStorage.clear();
    for (let key in localStorage) {
      if (localStorage.hasOwnProperty(key)) {
        localStorage.removeItem(key);
      }
    }

    navigate('/');
  }

```

Register:

```
GeneralContext.js U ×
client > src > context > GeneralContext.js > GeneralContextProvider > logout

  const inputs = {username, email, usertype, password, restaurantAddress, restaurantImage};

  const register = async () =>{
    try{
        await axios.post('http://localhost:6001/register', inputs)
        .then( async (res)=>{
            localStorage.setItem('userId', res.data._id);
            localStorage.setItem('userType', res.data.usertype);
            localStorage.setItem('username', res.data.username);
            localStorage.setItem('email', res.data.email);

            if(res.data.usertype === 'customer'){
                navigate('/');
            } else if(res.data.usertype === 'admin'){
                navigate('/admin');
            } else if(res.data.usertype === 'restaurant'){
                navigate('/restaurant');
            }
        }).catch((err) =>{
            alert("registration failed!!");
            console.log(err);
        });
    }catch(err){
        console.log(err);
    }
  }
```

All Products (User):

· Frontend

In the home page, we'll fetch all the products available in the platform along with the filters.

Fetching food items:

```
IndividualRestaurant.jsx 4, U
client > src > pages > customer > IndividualRestaurant.jsx > IndividualRestaurant > handleCategoryCheckBox

const fetchRestaurants = async() =>{
  await axios.get(`http://localhost:6001/fetch-restaurant/${id}`).then(
    (response)=>{
      setRestaurant(response.data);
      console.log(response.data)
    }
  ).catch((err)=>{
      console.log(err);
  })
}

const fetchCategories = async () =>{
  await axios.get('http://localhost:6001/fetch-categories').then(
    (response)=>{
      setAvailableCategories(response.data);
    }
  )
}

const fetchItems = async () =>{
  await axios.get(`http://localhost:6001/fetch-items`).then(
    (response)=>{
      setItems(response.data);
      setVisibleItems(response.data);
    }
  )
}
```

Filtering products:

```
Products.jsx 2, U
client > src > components > Products.jsx > Products > useEffect() callback

    const [sortFilter, setSortFilter] = useState('popularity');
    const [categoryFilter, setCategoryFilter] = useState([]);
    const [genderFilter, setGenderFilter] = useState([]);

    const handleCategoryCheckBox = (e) =>{
      const value = e.target.value;
      if(e.target.checked){
          setCategoryFilter([...categoryFilter, value]);
      }else{
          setCategoryFilter(categoryFilter.filter(size=> size !== value));
      }
    }

    const handleGenderCheckBox = (e) =>{
      const value = e.target.value;
      if(e.target.checked){
          setGenderFilter([...genderFilter, value]);
      }else{
          setGenderFilter(genderFilter.filter(size=> size !== value));
      }
    }

    const handleSortFilterChange = (e) =>{
      const value = e.target.value;
      setSortFilter(value);
      if(value === 'low-price'){
          setVisibleProducts(visibleProducts.sort((a,b)=>  a.price - b.price))
      } else if (value === 'high-price'){
          setVisibleProducts(visibleProducts.sort((a,b)=>  b.price - a.price))
      }else if (value === 'discount'){
          setVisibleProducts(visibleProducts.sort((a,b)=>  b.discount - a.discount))
      }
    }

    useEffect(()=>{

          if (categoryFilter.length > 0 && genderFilter.length > 0){
              setVisibleProducts(products.filter(product=> categoryFilter.includes(product.category) && genderFilter.includes(product.gender) ));
          }else if(categoryFilter.length === 0 && genderFilter.length > 0){
              setVisibleProducts(products.filter(product=> genderFilter.includes(product.gender) ));
          } else if(categoryFilter.length > 0 && genderFilter.length === 0){
              setVisibleProducts(products.filter(product=> categoryFilter.includes(product.category)));
          }else{
              setVisibleProducts(products);
          }

    }, [categoryFilter, genderFilter])
```

· **Backend**

In the backend, we fetch all the products and then filter them on the client side.

```javascript
// fetch products

app.get('/fetch-products', async(req, res)=>{
    try{
        const products = await Product.find();
        res.json(products);

    }catch(err){
        res.status(500).json({ message: 'Error occured' });
    }
})
```

Add product to cart:

· **Frontend**

Here, we can add the product to the cart and later can buy them.

```javascript
const handleAddToCart = async(foodItemId, foodItemName, restaurantId,
                                foodItemImg, price, discount) =>{
  await axios.post('http://localhost:6001/add-to-cart', {userId, foodItemId,
                                foodItemName, restaurantId, foodItemImg,
                                price, discount, quantity}).then(
      (response)=>{
          alert("product added to cart!!");
          setCartItem('');
          setQuantity(0);
          fetchCartCount();
      }
  ).catch((err)=>{
      alert("Operation failed!!");
  })
}
```

· **Backend**

Add product to cart:

```
// add cart item

app.post('/add-to-cart', async(req, res)=>{
    const {userId, foodItemId, foodItemName, restaurantId,
                foodItemImg, price, discount, quantity} = req.body
    try{
        const restaurant = await Restaurant.findById(restaurantId);
        const item = new Cart({userId, foodItemId, foodItemName,
                                restaurantId, restaurantName: restaurant.title,
                                foodItemImg, price, discount, quantity});
        await item.save();
        res.json({message: 'Added to cart'});
    }catch(err){
        res.status(500).json({message: "Error occured"});
    }
})
```

Order products:

Now, from the cart, let’s place the order

· Frontend

```
const placeOrder = async() =>{
  if(cart.length > 0){
    await axios.post('http://localhost:6001/place-cart-order', {userId, name,
                                mobile, email, address, pincode, paymentMethod,
                                orderDate: new Date()}).then(
      (response)=>{
        alert('Order placed!!');
        setName('');
        setMobile('');
        setEmail('');
        setAddress('');
        setPincode('');
        setPaymentMethod('');
        navigate('/profile');
      }
    )
  }
}
```

· **Backend**

In the backend, on receiving the request from the client, we then place the order for the products in the cart with the specific user Id.

index.js

server > index.js > then() callback > app.listen() callback

```javascript
// Order from cart

app.post('/place-cart-order', async(req, res)=>{
    const {userId, name, mobile, email, address, pincode,
                paymentMethod, orderDate} = req.body;
    try{
        const cartItems = await Cart.find({userId});
        cartItems.map(async (item)=>{
            const newOrder = new Orders({userId, name, email,
                        mobile, address, pincode, paymentMethod,
                        orderDate, restaurantId: item.restaurantId,
                        restaurantName: item.restaurantName,
                        foodItemId: item.foodItemId, foodItemName: item.foodItemName,
                        foodItemImg: item.foodItemImg, quantity: item.quantity,
                        price: item.price, discount: item.discount})
            await newOrder.save();
            await Cart.deleteOne({_id: item._id})
        })
        res.json({message: 'Order placed'});
    }catch(err){
        res.status(500).json({message: "Error occured"});
    }
})
```

Add new product:

Here, in the admin dashboard, we will add a new product.

· Frontend:

NewProduct.jsx 1, U

client > src > pages > restaurant > NewProduct.jsx > NewProduct

```
const handleNewProduct = async() =>{
  await axios.post('http://localhost:6001/add-new-product', {restaurantId: restaurant._id,
                productName, productDescription, productMainImg, productCategory, productMenuCategory,
                productNewCategory, productPrice, productDiscount}).then(
    (response)=>{
      alert("product added");
      setProductName('');
      setProductDescription('');
      setProductMainImg('');
      setProductCategory('');
      setProductMenuCategory('');
      setProductNewCategory('');
      setProductPrice(0);
      setProductDiscount(0);
      navigate('/restaurant-menu');
    }
  )
}
```

· **Backend:**

index.js

server > index.js > then() callback

```
    // Add new product
    app.post('/add-new-product', async(req, res)=>{
        const {restaurantId, productName, productDescription,
                    productMainImg, productCategory, productMenuCategory,
                    productNewCategory, productPrice, productDiscount} = req.body;
        try{
            if(productMenuCategory === 'new category'){
                const admin = await Admin.findOne();
                admin.categories.push(productNewCategory);
                await admin.save();
                const newProduct = new FoodItem({restaurantId, title: productName,
                            description: productDescription, itemImg: productMainImg,
                            category: productCategory, menuCategory: productNewCategory,
                            price: productPrice, discount: productDiscount, rating: 0});
                await newProduct.save();
                const restaurant = await Restaurant.findById(restaurantId);
                restaurant.menu.push(productNewCategory);
                await restaurant.save();
            } else{
                const newProduct = new FoodItem({restaurantId, title: productName,
                            description: productDescription, itemImg: productMainImg,
                            category: productCategory, menuCategory: productMenuCategory,
                            price: productPrice, discount: productDiscount, rating: 0});
                await newProduct.save();
            }
            res.json({message: "product added!!"});
        }catch(err){
            res.status(500).json({message: "Error occured"});
        }
    })
```

Along with this, implement additional features to view all orders, products, etc., in the admin dashboard.

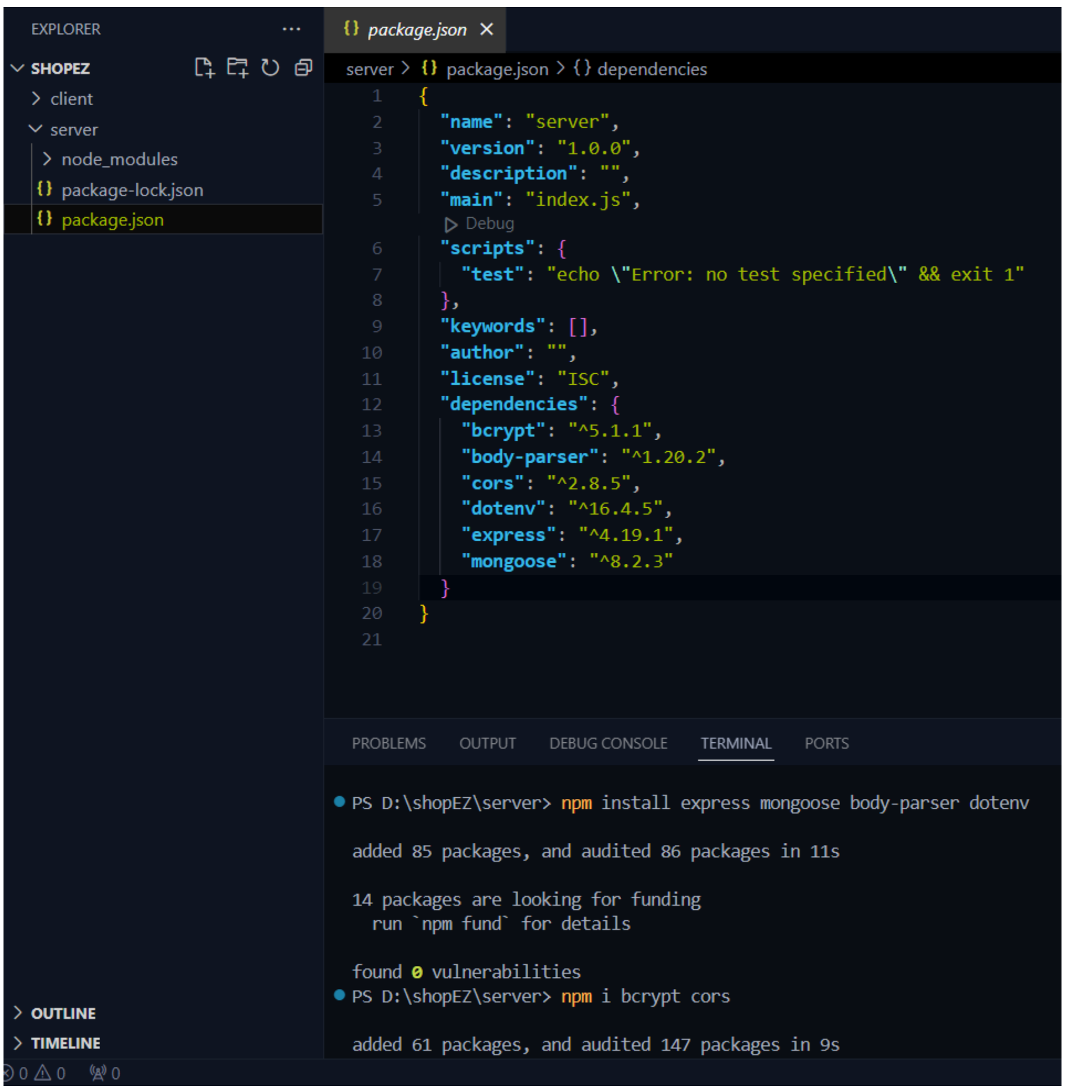
Backend

Set Up Project Structure:

• Create a new directory for your project and set up a package.json file using the npm init command.

• Install necessary dependencies such as Express.js, Mongoose, and other required packages.

Reference Image:



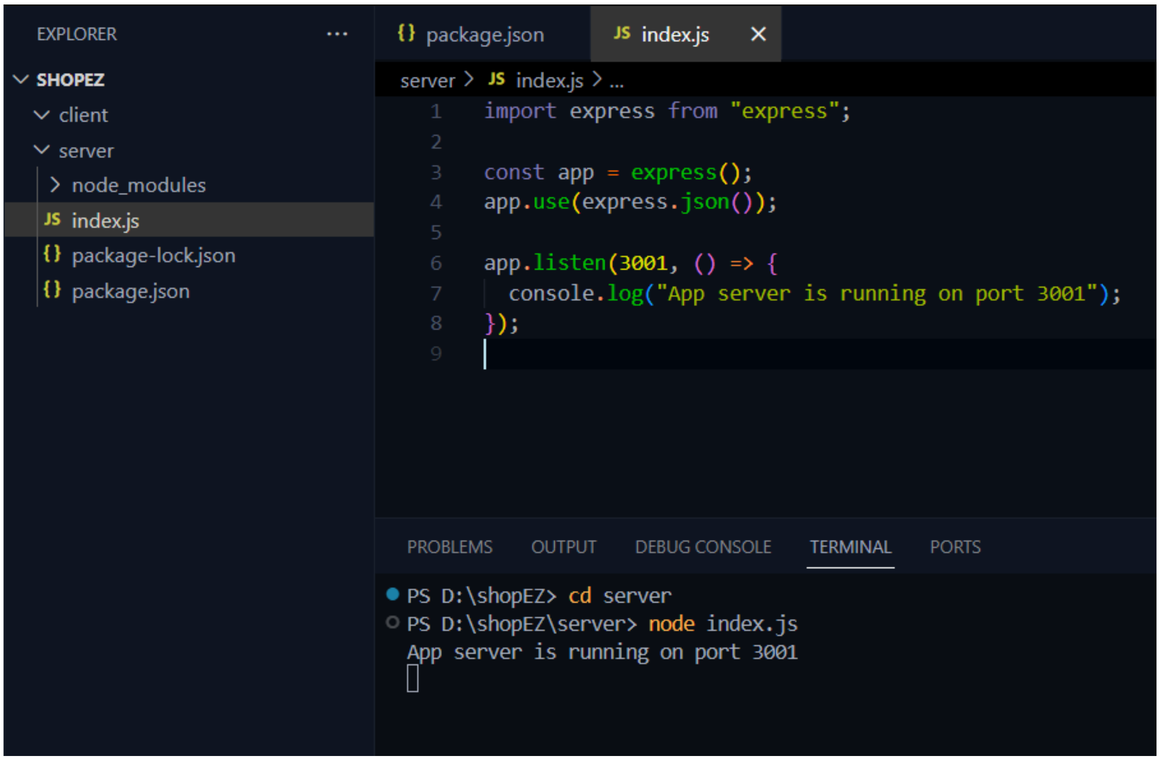
1. Setup express server:

• Create index.js file.

• Create an express server on your desired port number.

• Define API's

Reference Image:

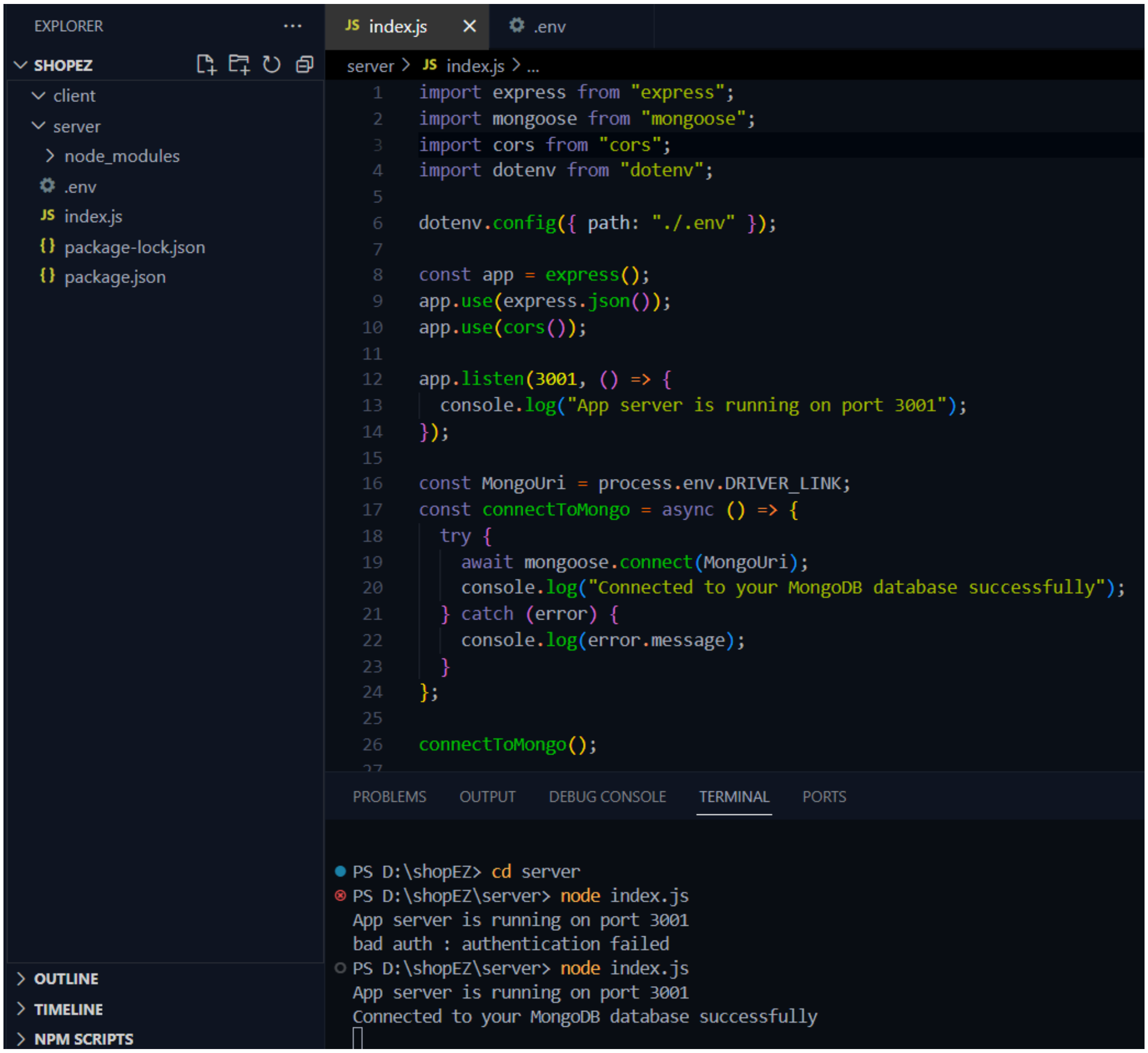


2. Database Configuration:

• Set up a MongoDB database either locally or using a cloud-based MongoDB service like MongoDB Atlas or use locally with MongoDB compass.

• Create a database and define the necessary collections for admin, users, restaurants, food products, orders,and other relevant data.

Reference Image:

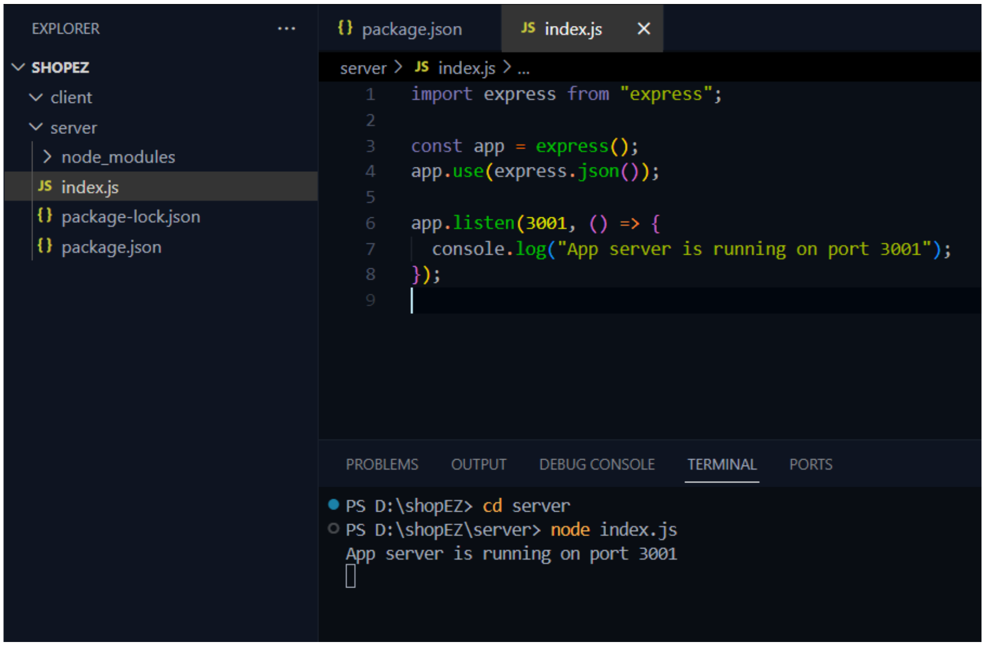


3. Create Express.js Server:

• Set up an Express.js server to handle HTTP requests and serve API endpoints.

• Configure middleware such as body-parser for parsing request bodies and cors for handling cross-origin requests.

Reference Image:



4. Define API Routes:

• Create separate route files for different API functionalities such as users, orders, and authentication.

• Define the necessary routes for listing products, handling user registration and login,managing orders, etc.

• Implement route handlers using Express.js to handle requests and interact with the database.

5. Implement Data Models:

• Define Mongoose schemas for the different data entities like products, users, and orders.

• Create corresponding Mongoose models to interact with the MongoDB database.

• Implement CRUD operations (Create, Read, Update, Delete) for each model to perform database operations.

6. User Authentication:

• Create routes and middleware for user registration, login, and logout.

• Set up authentication middleware to protect routes that require user authentication.

7. Handle new products and Orders:

• Create routes and controllers to handle new product listings, including fetching products data from the database and sending it as a response.

• Implement ordering(buy) functionality by creating routes and controllers to handle order requests, including validation and database updates.

8. Admin Functionality:

• Implement routes and controllers specific to admin functionalities such as adding products, managing user orders, etc.

• Add necessary authentication and authorization checks to ensure only authorized admins can access these routes.

9. Error Handling:

• Implement error handling middleware to catch and handle any errors that occur during the API requests.

• Return appropriate error responses with relevant error messages and HTTP status codes.

Database

Uses MongoDB for storing all structured data. Below are the key schemas used:

- userSchema: User details (username, email, password).

- productSchema: Product information (name, price, description).

- ordersSchema: Order records (userId, productId, quantity, size, date).

- cartSchema: Temporary cart data linked to a user.

- adminSchema: Admin data (categories, promoted restaurants).

- restaurantSchema: Restaurant data including profile and menu.

Setup Instructions

Prerequisites

- Node.js (v14+)

- MongoDB (Compass or Atlas)

- npm or yarn

Installation

- Create project folder and initialize package.json

- Install Express.js, Mongoose, dotenv, cors, body-parser

- Create index.js and set up Express server

- Configure MongoDB and environment variables

Folder Structure

Client

- src/components: React components for UI

- src/pages: Individual pages (Home, Cart, Checkout)

- src/context: State management using Context API

Server

- server/index: Route handlers for users, products, orders, admin

- server/Schema: Mongoose schemas and models

- server/index: Logic for handling API calls

- server/: Authentication and error handling logic

Running the Application

The project uses concurrently to streamline development by running both frontend and backend servers in parallel from the root directory.

To start the full stack application:

➔ npm start

| This command triggers scripts defined in the root package.json, simultaneously launching:

- **Frontend**: React app located in /client
- **Backend**: Express server located in /server

No need to navigate into individual folders manually—everything is wired up for convenience and efficiency.

API Documentation

Method	Endpoint	Description
POST	/api/register	Register a new user with email and password. performs validation and hashes passwords using ɔ byscript.
POST	/api/login	Authenticates a user and returns a JWT token.
GET	/api/products	Retrieves a list of available products. Supports filtering and pagination (optional).
POST	/api/orders	Submits a new order. Requires authentication.
GET	/api/orders/:id	Fetches details for a specific order by ID. Secured accessible by order owner or admin.

Authentication

User authentication is managed via **JSON Web Tokens (JWT)**. The backend issues a token upon successful login, which must be included in the Authorization header of protected requests.

Access Control:

- Regular users: Can view products, create orders, and view their own orders.
- Admins: Have extended privileges like viewing all orders and managing products.

User Interface

The frontend is built with **React**, focusing on simplicity and responsiveness. Key features include:

- **Navigation**: Seamless transitions between views using React Router.
- **Pages**:

▪ Home
▪ Resturant Listing
▪ Food items
▪ Cart
▪ Checkout
▪ Admin/User Dashboards

The layout is responsive and styled with **Bootstrap**, ensuring optimal experience across devices.

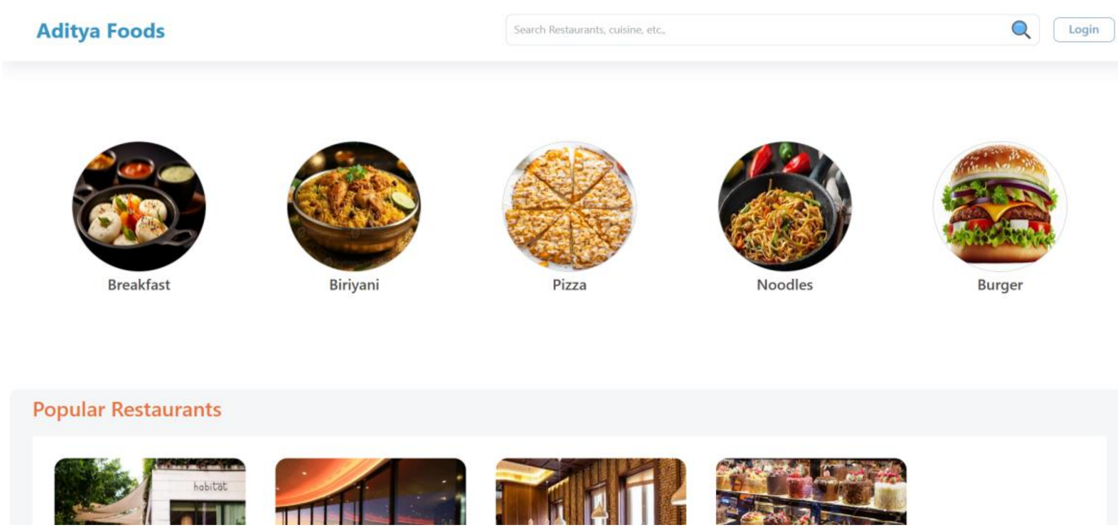
Testing

- **Manual Testing**: Conducted during development by interacting with the frontend and verifying backend responses.
- **Postman**: Used to test and document APIs, verify headers, request bodies, and token validity.

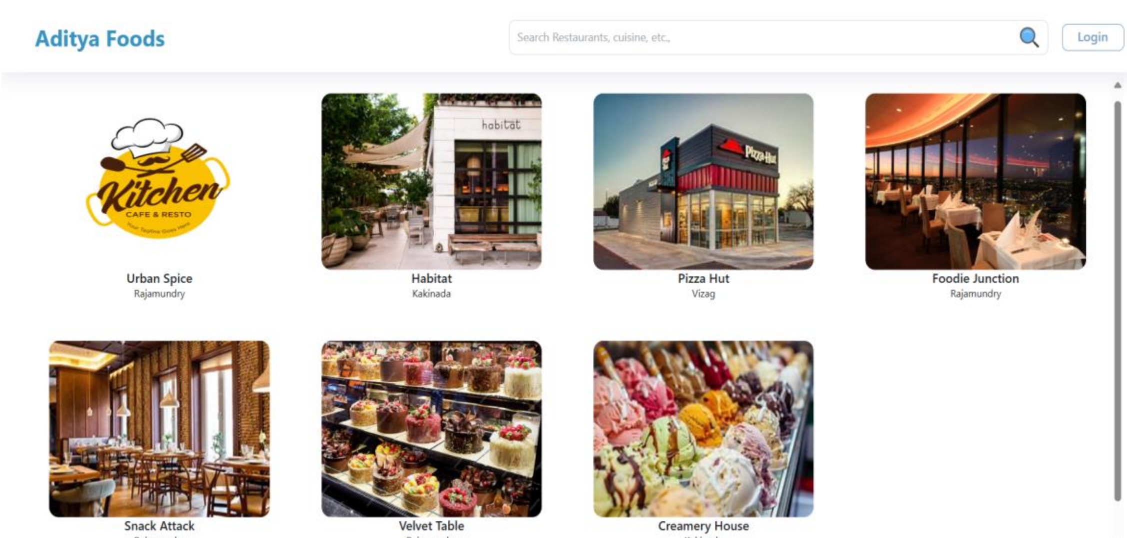
Future enhancement: Add unit and integration tests using frameworks like Jest or Mocha.

Screenshots or Demo

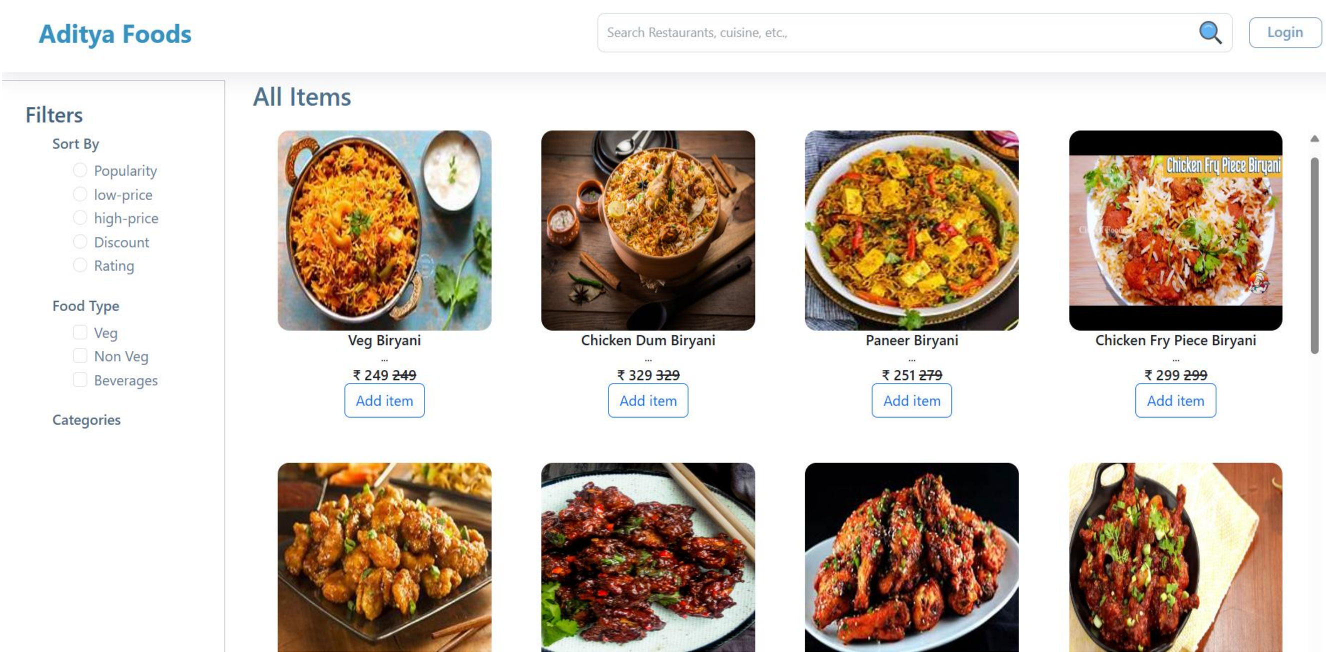
- **Landing page**



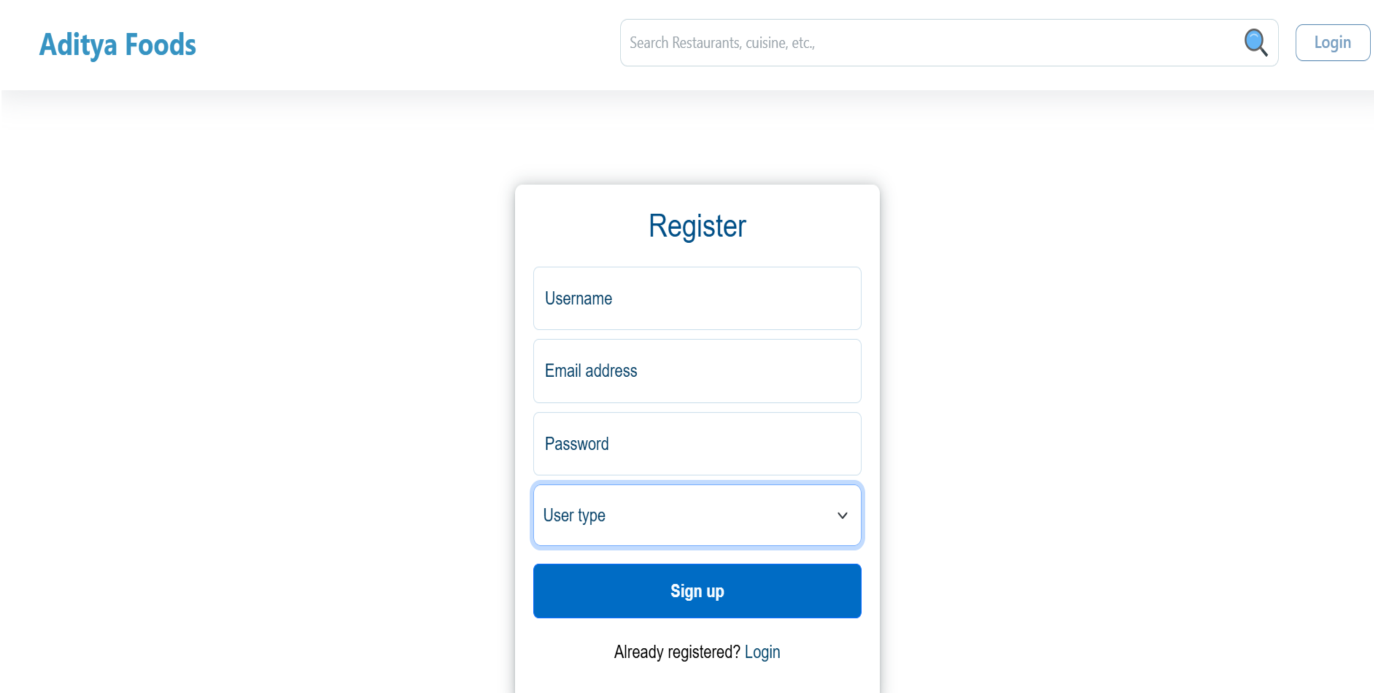
- **Restaurant**



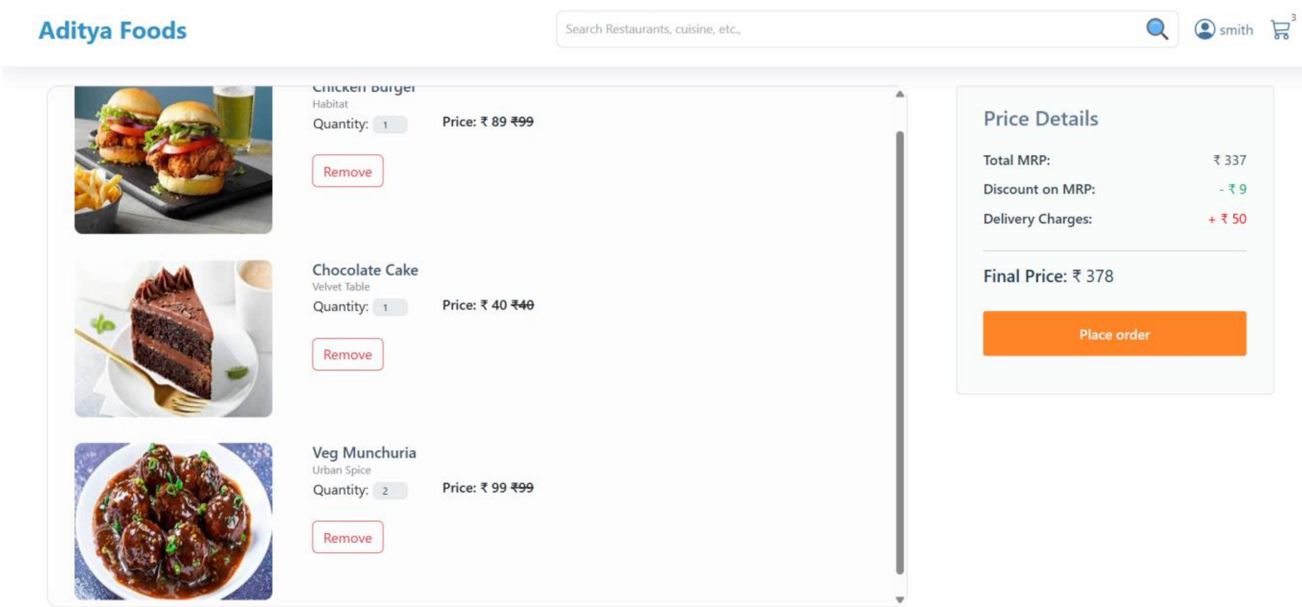
- **Restaurant Menu**



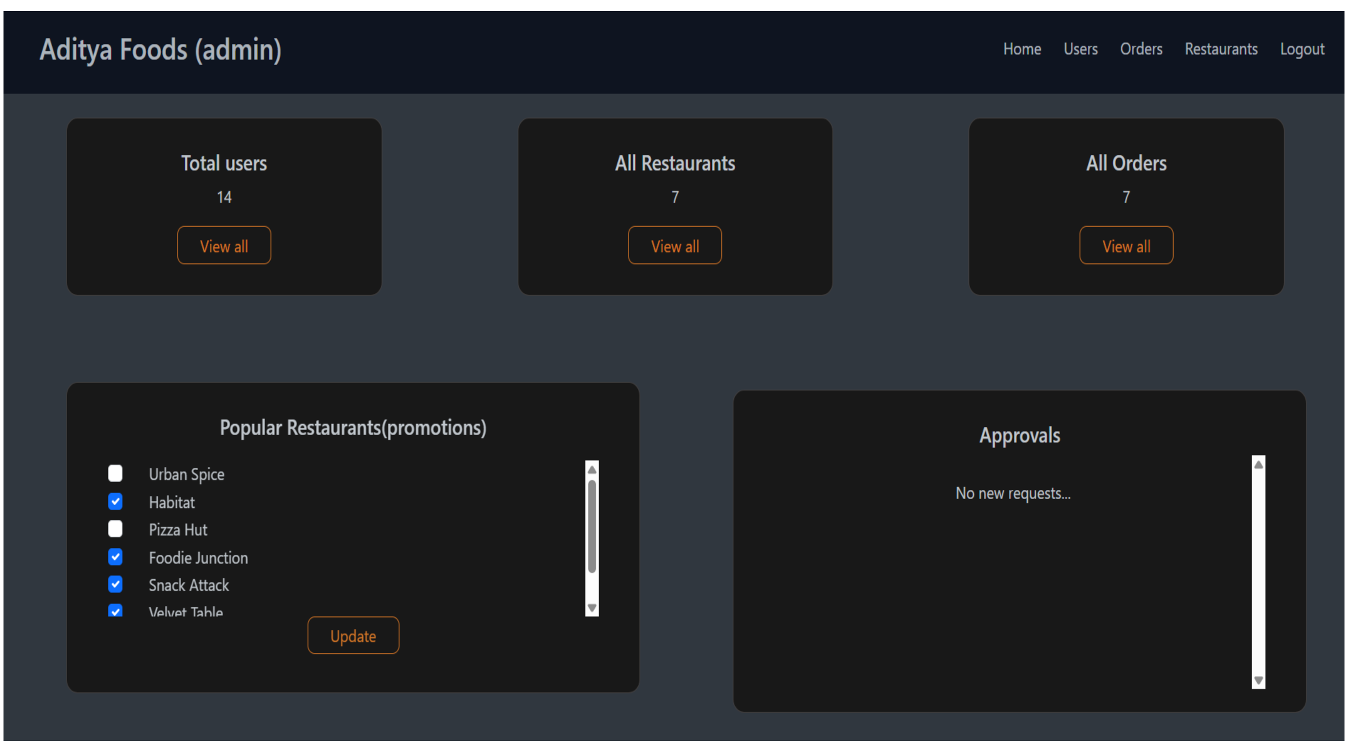
- **Authentication**



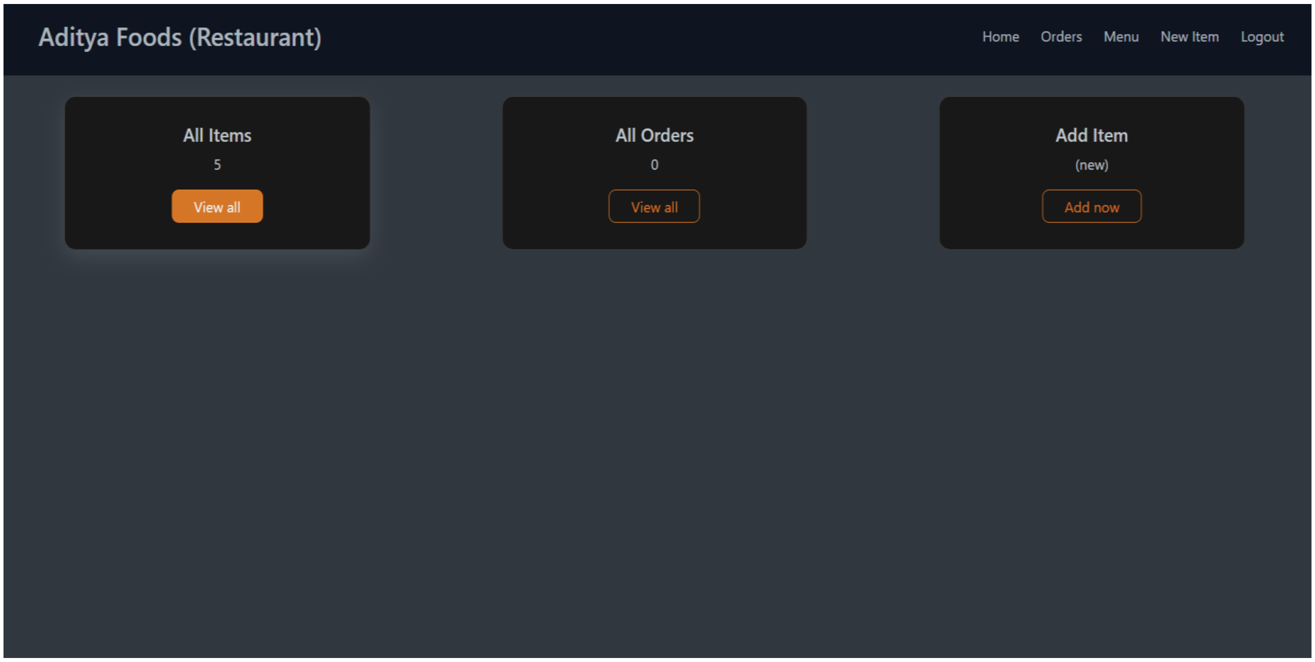
- **User Profile & Cart**



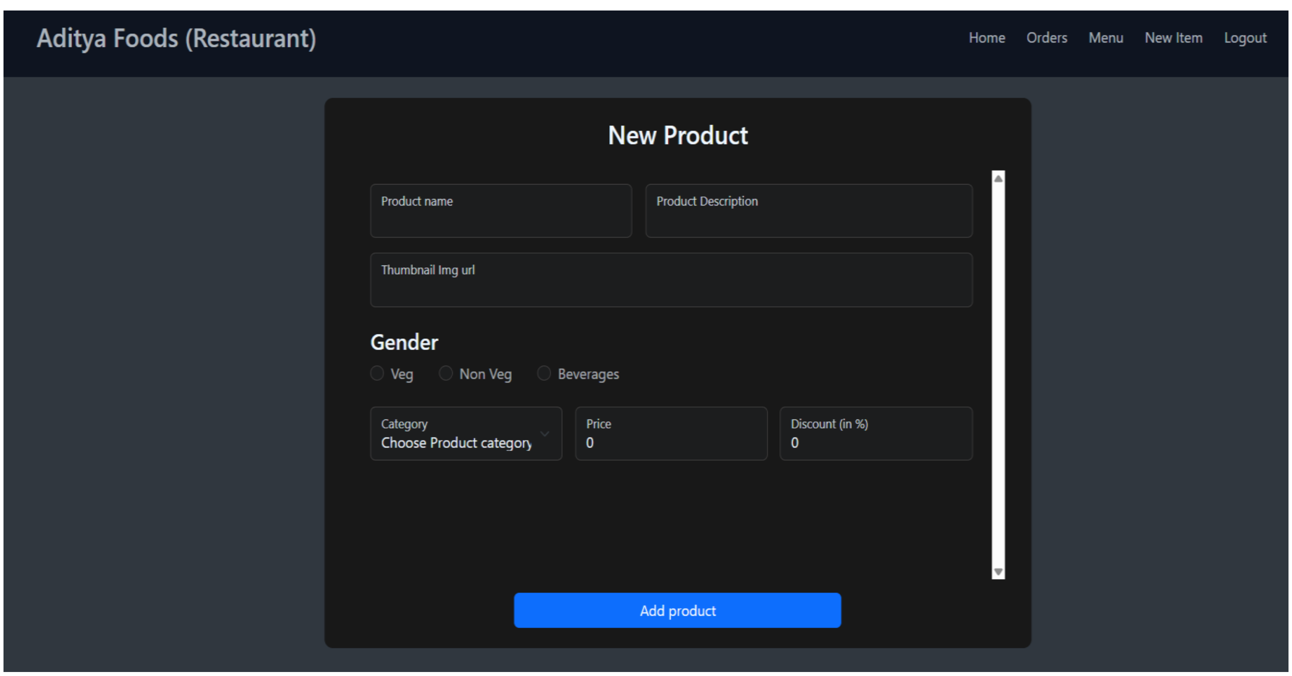
- **Admin dashboard**



- **Restaurant Dashboard**



- **New Item**



Demo Video :

https://github.com/ksatyasai/Food-Ordering-Website-Mern/tree/main/Video%20doc

Known Issues

- Lack of automated testing (unit/integration).
- No continuous integration or deployment pipeline in place.
- Error handling and form validation can be improved further.
- Product and order data are not paginated.

Future Enhancements

- Integrate **payment gateway** (e.g., Stripe, Razorpay) for secure transactions.
- Implement **real-time order status updates** using WebSockets (e.g., Socket.IO).
- Add **push notifications** for users on order activity.
- Set up **CI/CD pipeline** for automated deployment and testing.
- Improve **admin panel** with analytics and product insights.
- Add **user profile management** (update info, view past orders).